Manipulación y operaciones con arrays

Asignación de arrays

Cuando realizamos la asignación entre dos variables, lo que ocurre es que el contenido de la variable de la derecha "se copia" izquierda en la variable de la izquierda, a partir de ese momento tenemos dos variables totalmente independientes, cuyo valores dependen de las operaciones en las que intervengan:

Ejemplo

```
<script>
        let variable1 = 3;
        let variable2 = variable1;
        variable2 = variable2 * 2;
        console.log("variable1=" + variable1);
        console.log("variable2=" + variable2);
</script>
```

```
variable1=3
variable2=6
```

¿Qué ocurre si hacemos esto mismo con arrays?

Ejemplo

```
<script>
      let preciosSinIVA=[20,100,5];
      let preciosConIVA=preciosSinIVA;
      preciosConIVA[0]=preciosConIVA[0]*1.16;


      console.log("precios sin IVA: "+preciosSinIVA);
      console.log("precios con IVA: "+preciosConIVA);
    </script>
```

```
precios sin IVA: 23.2,100,5
precios con IVA: 23.2,100,5
```

Como vemos se ha modificado el primer elemento de los dos arrays. ¿Por qué? Porque el identificador de un array no es donde se almacenan los datos, sino que es una referencia que apunta a la estructura de datos, por eso, cuando asignamos identificadores, estamos indicando que los dos identificadores apuntan a la misma estructura de datos, de manera que cuando se realiza una modificación sobre cualquiera de los identificadores, ésta afectará a la misma estructura de datos.

Añadir elementos a un array

Tenemos varias maneras de añadir un nuevo elemento a un array:

1. Si sabemos cuántos elementos tiene el array, basta con indicar un elemento al final:

```
<script>
    let preciosSinIVA = [20, 100, 5];
    preciosSinIVA[3] = 80;
    console.log("precios sin IVA: " + preciosSinIVA);
  </script>
```

```
precios sin IVA: 20,100,5,80
```

2. Si no conocemos el tamaño del array, usaremos el método `push()` que añade el nuevo elemento en la última posición:

```
<script>
    let preciosSinIVA = [20, 100, 5];
    preciosSinIVA.push(80);
    console.log("precios sin IVA: " + preciosSinIVA);
  </script>
```

```
precios sin IVA: 20,100,5,80
```

Podemos añadir varios elementos a la vez:

```
preciosSinIVA.push(80,90,100);
```

```
precios sin IVA: 20,100,5,80,90,100
```

3. Si queremos añadir un elemento en la primera posición usaremos el método unshift():

```
<script>
    let preciosSinIVA = [20, 100, 5];
    preciosSinIVA.unshift(80);
    console.log("precios sin IVA: " + preciosSinIVA);
  </script>
```

```
precios sin IVA: 80,20,100,5
```

Podemos añadir varios elementos a la vez:

```
preciosSinIVA.unshift(80,90,100);
```

```
precios sin IVA: 80,90,100,20,100,5
```

Ejemplo: Vamos a crear un array vacío llamado "vector" e insertaremos el cuadrado de 5 números aleatorios entre 0 y 10. Luego mostraremos el resultado en pantalla

```
<script>
  let vector=new Array(5);
  for(let i=0;i<5;i++)
  {
    vector[i]=Math.floor(Math.random()*10)**2;
  }
  for(let i=0;i<5;i++)
  {
    document.write(vector[i]+"<br/>");
  }

</script>
```

Eliminar elementos de un array

- Si queremos eliminar el primer elemento de un array usaremos el método `shift()`;

```
preciosSinIVA.shift();
```

- Si queremos eliminar el último elemento de un array usaremos el método `pop()`:

```
preciosSinIVA.pop();
```

En ambos casos se devolverá el el elemento eliminado, si no se ha eliminado ningún elemento, se devolverá `undefined`:

```
let preciosSinIVA = [80,20, 100, 5];
var eliminado=preciosSinIVA.shift();
console.log("Se ha eliminado el elemento: " + eliminado);
```

```
Se ha eliminado el elemento: 80
```

- Al modificar la propiedad `length`, se eliminan los elementos que se quedan fuera de esa longitud:

```
let preciosSinIVA = [80,20, 100, 5];
preciosSinIVA.length=2;
console.log("el array: " + preciosSinIVA);
```

```
el array: 80,20
```

- También podemos eliminar elementos de un array indicando desde qué posición queremos eliminar y cuántos elementos usando el método `splice()`

```
let preciosSinIVA = [80,20, 100, 5];
preciosSinIVA.splice(1,2);//eliminamos desde la posición 1, dos elementos
console.log("el array: " + preciosSinIVA);
```

```
el array: 80,5
```

Concatenación de arrays

Podemos añadir al final de un array otro array, usando el método `concat`, ninguno de los dos arrays será modificado, sino que se creará uno nuevo con el contenido de ambos. El array que aparecerá en primer lugar será el que haga uso de `concat`:

```
let vector1=[1,5.2,"María",false];
let vector2=[10,9,8,10];
let nuevo=vector1.concat(vector2);
console.log(nuevo);
```

```
[1, 5.2, 'María', false, 10, 9, 8, 10]
```

Copia de arrays

Podemos copiar arrays completos o solo una parte, usando el método `slice()`.

- Si no se indican parámetros, se copia todo el contenido de un array en otro:

```
let vector1=[1,5.2,"María",false];
let copiaVector1=vector1.slice();
console.log(copiaVector1);
```

```
(4) [1, 5.2, 'María', false]
```

- Si queremos copiar solo una parte, indicaremos 2 parámetros: la posición desde la que se desea copiar (incluido el elemento que ocupa esa posición) y el segundo será la posición hasta la que se quiere copiar (no incluido el elemento que ocupa esa posición)

```
let vector1=[1,5.2,"María",false];
let copiaVector1=vector1.slice(2,3);
console.log(copiaVector1);
```

```
['María']
```

- Si queremos copiar todos los elementos a partir de una posición, indicaremos solo un parámetro:

```
let vector1=[1,5.2,"María",false];
let copiaVector1=vector1.slice(2);
console.log(copiaVector1);
```

```
['María', false]
```

Buscar elementos en un array

El método `IndexOf()` nos devuelve la posición que ocupa el primer elemento que buscamos si lo encuentra o -1 si no lo encuentra. También podemos indicar a partir de qué posición queremos buscar:

```
let vector=[1,2,1,5,6,8];
let resultado=vector.indexOf(1);
console.log("El 1 aparece por primera vez en la posición "+resultado);
```

```
El 1 aparece por primera vez en la posición 0
```

```
let vector=[1,2,1,5,6,8];
let resultado=vector.indexOf(1,2);//en qué posición aparece el 1 a partir de la posición 2
console.log("El 1 aparece por primera vez en la posición "+resultado);
```

```
El 1 aparece por primera vez en la posición 2
```

El método `lastIndexOf()` realiza las mismas tareas que `indexOf()` pero trabajando desde el extremo derecho del array:

```
let vector=[1,2,1,5,6,8];
let resultado=vector.lastIndexOf(1);
console.log("Desde el final del array 1 aparece por primera vez en la posición "+resultado);
```

```
Desde el final del array 1 aparece por primera vez en la posición 2
```

```
let vector=[1,2,1,5,6,8];
let resultado=vector.lastIndexOf(1,1);
console.log("Desde la posición 1 hacia la izquieda, el 1 aparece por primera vez en la posición "+resultado);
```

```
Desde la posición 1 hacia la izquieda, el 1 aparece por primera vez en la posición 0
```

El método `includes`, nos indica si el elemento existe o no en el array, devolviendo `true` si hay al menos una coincidencia o `false` si no la hay:

```
let vector = [1, 2, 1, 5, 6, 8];
let resultado = vector.includes(1);
console.log(resultado);
```

```
true
```

Ordenación de arrays

El método `sort()` permite ordenar un array teniendo en cuenta el lugar que ocupa el caracter en la tabla Unicode: https://home.unicode.org/

```
let vector = [1, 2, 1, 5, 6, 8];
vector.sort();
console.log(vector);
```

```
[1, 1, 2, 5, 6, 8]
```

```
let vector = ["Dado","dado","Cruz","cara"];
vector.sort();
console.log(vector);
```

```
['Cruz', 'Dado', 'cara', 'dado']
```

El método `reserve()` nos permite darle la vuelta a un array, es decir, invertirlo:

```
let vector = ["Ana","Pedro","Belén","Juan"];
vector.reverse();
console.log(vector);
```

```
['Juan', 'Belén', 'Pedro', 'Ana']
```